

Solving MAF Using ILP With Trios and Quartets

Juni Weisteen Bjerde Wim Van den Broeck Krishnan Dehaleesan
Yash Hiren More Jakob Rødal Skaar Tomáš Turek Sam Urmian

July 2026

1 Preliminaries

We consider the Maximum Agreement Forest (MAF) problem. The problem takes as input a set of rooted phylogenetic trees $\mathcal{T} = \{T_1, T_2, \dots, T_t\}$. Each tree T_i has the same leaf-set X of size $|X| = n$. An agreement forest for $\mathcal{T}$ is a forest $\mathcal{F} = \{F_1, F_2, \dots, F_r\}$ of trees such that $\mathcal{F}$ can be obtained from each T_i by first removing a set of edges and then doing some clean-up by exhaustively contracting inner vertices of degree at most 2. The goal of the MAF-problem is to find an agreement forest $\mathcal{F} = \{F_1, F_2, \dots, F_r\}$ of minimum size, i.e. for minimum value of r .

We will now define some simple notation for a single rooted tree $T = (V, E)$ with X as its leaf-set, where ρ denotes the root vertex. For two vertices $u, v \in V$ we say that u lies below v , denoted by $u < v$, if v lies on the unique $\rho - u$ path. For any subset $S \subseteq V$ we define the *lowest common ancestor* (LCA) of the set as the smallest $v \in V$ for which $\forall s \in S : s \leq v$, i.e. no other such vertex $u \in V, u < v$ exists. We denote this as $\text{lca}(S)$. When we talk about sets of specific elements, for example $S = \{u, v\}$, then we abbreviate our notation to $\text{lca}(a, b)$ only.

Let $Y \subseteq X$. We define $T[Y]$ to be the smallest subtree of T that includes all leafs in Y . Next, $T|_Y$ denotes the tree $T[Y]$ with forced contractions.

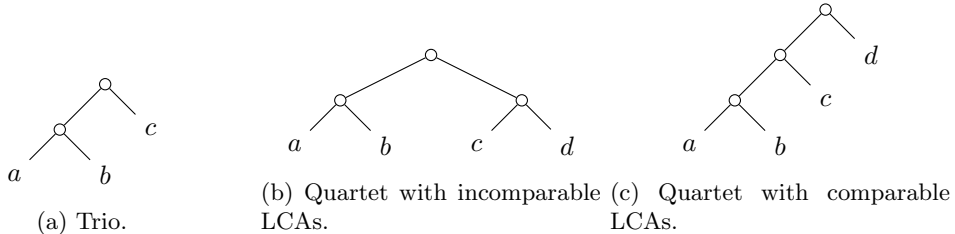
2 Integer Linear Program

We use the ILP defined by Wu [Wu09]. We reuse the notation of *trios* from [DvJZ25], because they use similar constraints, and also add the term *quartets*.

To speed up the implementation we use LCA to classify all cases in constant time if we precompute the LCA table for all pairs of nodes.

We look at trios denoted as $a, b|c$ as shown in the picture 1a. Observe that this is the only constellation in which three leafs $R = \{a, b, c\}$ can encounter in $T|_R$, unless the labels are switched. Also see that $a, b|c$ and $b, a|c$ has the same properties hence we do not distinguish them.

Next look at quartets, here we have more options because we only care if the paths intersect or not. We will denote them as $a, b|c, d$. We may see two examples of such occurrences on Figure 1b and Figure 1c.



2.1 Incompatible Trios

A trio of leaves $\{a, b, c\}$ is an incompatible trio if in T_1 we have $a, b|c$ and in some T_j , for $1 < j \leq t$ we have either $a, c|b$ or $b, c|a$. Note that cutting one of the edges in $T_1[\{a, b, c\}]$ is enough to resolve

this incompatible trio. This is due to the fact that one edge will remove one of a, b or c from the rest. Then the trio on itself would be fine with the definition of MAF.

It is easy to find such occurrence of incompatible trios in our input. First, we go over all triples of leafs a, b, c , in $\mathcal{O}(n^3)$ time, and find such triple that suffices the following inequality. $\text{lca}_1(a, b) \neq \text{lca}_1(a, b, c)$. Since easily seen it must be $\text{lca}_1(a, b) \leq \text{lca}_1(a, b, c)$, then when inequality occurs it is our trio. Then, we go through all other trees and if there is trio for which $\text{lca}_j(a, b) = \text{lca}_j(a, b, c)$ we have found incompatible trio.

So we gather all trios into the set of $\mathcal{R}$ and we define $\mathcal{L}(R)$ to be $T_1[R]$, for $R \in \mathcal{R}$.

2.2 Incompatible Quartets

Now, we look at quartet. It is a bit harder to find such quartet by using LCAs only. The definition is based on the intersection of $a - b$ and $c - d$ paths.

Firstly, it must be true that $\text{lca}_1(a, b) \neq \text{lca}_2(c, d)$. Otherwise their paths intersect. Next we split into two cases.

1. First, if $\text{lca}_1(a, b, c) = \text{lca}_1(a, b)$ then we check that $\text{lca}_1(c, d)$ is not the same as $\text{lca}_1(x, y)$ for $x \in \{a, b\}$ and $y \in \{c, d\}$.
2. Second, that the equality does not hold. Then we check that $\text{lca}_1(a, b)$ is not the same as $\text{lca}_1(x, y)$ for x, y same as before.

Proof sketch. The very first check is clear. Next in the first branch we get that c is below a, b tree and then d must be also below a, b tree and then the further checks satisfy this condition. In the other case we have that a is below c, d tree or they are completely disjoint. In the former case it is enough to check the values and in the latter case we actually do not need any further checks.

Therefore we find quartets that satisfy this for the first tree and whenever we find a different tree for which this does not hold we say it is incompatible quartet. We collect all such quartets into $\mathcal{Q}$ and for each $Q \in \mathcal{Q}$ we define $\mathcal{L}(Q)$ as $T[\{a, b\}] \cup T[\{c, d\}]$.

By using these definitions we create our ILP. And from [Wu09] we get that the optimal integer solution is actually a Maximum Agreement Forest as desired.

$$\begin{aligned}
& \text{Minimize: } \sum_{e \in E(T_1)} x_e \\
& \text{Subject to: } \sum_{e \in \mathcal{L}(R)} x_e \geq 1 & \forall R \in \mathcal{R} \\
& \sum_{e \in \mathcal{L}(Q)} x_e \geq 1 & \forall Q \in \mathcal{Q} \\
& x_e \in \{0, 1\} & \forall e \in E(T_1)
\end{aligned}$$

3 Reductions and Speed-Ups

We implement simple and known reduction to reduce the size of the input trees. We call $a, b \in X$ a *cherry* (fig. 2) if it a and b shares the same parent and this is in every tree from the input. Then we can reduce this by substituting their parent with a and remember that b is also included. Notably one cherry could lead to creating another cherry. But this can be done in linear time.

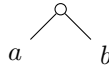


Figure 2: A cherry.

Moreover, we add additional constraints to help the ILP solver to solve it faster. We add only one type of constraints. We call them *forks*, see Figure 3. We first observe that deleting two edges

is always enough because the third is implicitly contracted. Moreover it is safe to say that if we remove exactly two edges then it is always the bottom two. In ILP formulation we may say that $2a + b + c \leq 2$.



Figure 3: A simple fork.

Another very straightforward constraint is that the sum of all edges in T_1 is at most $n - 2$. That is due to the fact that there is always a simple solution where every tree in MAF is a node except for one simple two-leaf tree. Moreover, we can have a lower bound for this as well if we have some, possibly infeasible, MAF.

We have added at most $\mathcal{O}(n)$ additional constraints, which is not much. On the other hand we can have up-to $\mathcal{O}(n^3 + n^4)$ constraints for trios and quartets, which is still polynomial, but not really nice. As it turns out it is rarely the case. But we can still do better by choosing only some constraints and then iteratively add new ones and solve the new ILP. Therefore we sort all trios and quartets based the number of edges in it in an increasing way. Then we establish counters for leafs and set them to $\max\{2, \log_2(\text{number of trios (or quartets)})\}$ and when we are looking at a trio (or quartet) in a length-increasing order we check counters for each leaf and at least one of them must be non-zero. Then we reduce all counters by one to 0. In the next iterations we always have some forest and possibly unsatisfied constraints. We go through all the constraints and if (1) there is no unsatisfied constraint then we are done, otherwise (2) we add constraints by using the same procedure as before, i.e., leaf counters. This serves as a lower bound for the very last constraint summing all edges.

Lastly we also have a heuristic approach that is greedily keeping found cut edges from the previous iterations. Therefore this is usually much faster and it is a way how to obtain a feasible solution (which is not necessarily optimal). Then in the exact solving we may use this feasible solution as an upper bound and also to warm-start the ILP with feasible solution.

References

- [DvJZ25] Jordan Dempsey, Leo van Iersel, Mark Jones, and Norbert Zeh. A simple 4-approximation algorithm for maximum agreement forests on multiple unrooted binary trees. *Information Processing Letters*, 190:106572, August 2025.
- [Wu09] Yufeng Wu. A practical method for exact computation of subtree prune and regraft distance. *Bioinformatics*, 25(2):190–196, January 2009.